Traffic AI — Project Overview

Traffic AI — Full Project Overview

Directory Structure

```
traffic_ai/
├── requirements.txt
├── run.sh
├── models/
│   ├── __init__.py
│   ├── markov_api.py       → port 8001
│   ├── rf_api.py           → port 8002
│   ├── lstm_api.py         → port 8003
│   └── bayesian_api.py     → port 8004
└── ensemble/
    ├── __init__.py
    ├── context_detector.py
    ├── ensemble_engine.py
    ├── tomtom_fetcher.py
    ├── main_api.py         → port 8000
    └── static/
        └── index.html      (interactive web UI)
```

High-Level Architecture

A **multi-model ensemble system** for real-time traffic congestion prediction. Four independent ML models each expose a FastAPI microservice. A central orchestrator detects the current traffic context, assigns weights to each model accordingly, and fuses their probability outputs into a single prediction.

```
Map click / API call
        │
        ▼
 ensemble/main_api.py  (port 8000)
        │
        ├──[context_detector]── detect context from hour, weather, accident
        │
        ├──[tomtom_fetcher]─── fetch live speed/weather via TomTom + Open-Meteo
        │
        ├── async fan-out to all 4 models:
        │       markov_api        :8001  /predict/markov
        │       rf_api            :8002  /predict/randomforest
        │       lstm_api          :8003  /predict/lstm
        │       bayesian_api      :8004  /predict/bayesian
        │
        └──[ensemble_engine]─── weighted fusion → predicted state + confidence
```

Services

`models/markov_api.py` — port 8001

- Hardcoded 4×4 **Markov transition matrix** (states: free → moderate → heavy → jam)

- Adjusts row probabilities at inference time based on speed ratio, vehicle density, and adverse weather
- No training required; stateless

`models/rf_api.py` — port 8002

- **RandomForestClassifier** (sklearn, 150 trees, depth 10)
- Trained at startup on 1000 synthetic samples that encode domain congestion rules
- Features: `[speed, vehicle_count, hour, weather_encoded, previous_state]`

`models/lstm_api.py` — port 8003

- 1-layer **LSTM** (PyTorch, hidden=32) trained for 40 epochs at startup on 600 synthetic sequences of length 6
- Features per step: `[speed_norm, density_norm, hour_sin, hour_cos, weather_norm]`
- At inference, builds an artificial 6-step sequence with small Gaussian jitter from the single input snapshot

`models/bayesian_api.py` — port 8004

- **Bayesian Network** (pgmpy) with structure: `Weather → Congestion ← TimeOfDay ← Density`
- 27-column hand-crafted CPT encoding domain knowledge
- Uses Variable Elimination at inference; inputs discretized into 3-level categories

---

Ensemble Layer

`ensemble/context_detector.py`

Detects one of 6 contexts (priority order):

1. `ACCIDENT_EVENT` — if `accident=True`
2. `WEATHER_EVENT` — if weather in `{rain, snow, fog}`
3. `MORNING_RUSH` — hour 7–10
4. `EVENING_RUSH` — hour 17–20
5. `NIGHT_LOW_TRAFFIC` — hour 0–5
6. `NORMAL_CONDITIONS` — default

`ensemble/ensemble_engine.py`

Context-specific weight tables (sum to 1.0):

Context	LSTM	RF	Markov	Bayesian
Morning/Evening Rush	0.5	0.2	0.2	0.1
Night	0.1	0.3	0.5	0.1
Weather Event	0.2	0.2	0.1	0.5
Accident	0.2	0.5	0.1	0.2
Normal	0.3	0.4	0.2	0.1

`fuse()` computes weighted average over 4 probability vectors → argmax = predicted state; confidence = top-2 prob sum.

`ensemble/tomtom_fetcher.py`

- Calls **TomTom Flow Segment API** for live speed + free-flow speed
- Calls **Open-Meteo** (free, no key) for weather via WMO codes
- Calls **TomTom Reverse Geocode** for street name
- Graceful fallback to deterministic synthetic data (seeded by lat/lon) if `TOMTOM_API_KEY` is unset

`ensemble/main_api.py` — port 8000

Three prediction endpoints:

- `POST /predict_traffic` — manual JSON input
- `GET  /fetch_location?lat=&lon=` — raw live data from map click
- `POST /predict_from_map?lat=&lon=` — one-click: fetches live data then runs full ensemble
- `GET  /` — serves the interactive HTML/JS map UI from `static/index.html`

Generates a **natural language summary** of the result (which model led, why, confidence word, road conditions).

Request / Response

Input (`POST /predict_traffic`)

```
{
  "road_id": 142,
  "avg_speed": 24,
  "vehicle_count": 210,
  "weather": "rain",
  "hour": 8,
  "previous_state": 2,
  "accident": false
}
```

Output

```
{
  "road_id": 142,
  "timestamp": "2026-03-06T08:00:00+00:00",
  "context": "weather_event",
  "model_predictions": {
    "markov":        [0.05, 0.20, 0.50, 0.25],
    "random_forest": [0.03, 0.15, 0.52, 0.30],
    "lstm":          [0.04, 0.18, 0.48, 0.30],
    "bayesian":      [0.06, 0.22, 0.44, 0.28]
  },
  "model_weights":        { "bayesian": 0.5, "random_forest": 0.2, "lstm": 0.2, "markov":
        0.1 },
  "final_probabilities":  { "free": 0.05, "moderate": 0.20, "heavy": 0.45, "jam": 0.30 },
  "predicted_state":      "heavy",
  "confidence":           0.75,
```

```
  "natural_language_summary": "At 8:00 AM, Road 142 is experiencing adverse weather
        conditions. Weather is rain, which is affecting road conditions. Current average
        speed is 24 km/h with 210 vehicles observed. The AI predicts that traffic is heavily
        congested — significant delays likely. The system is fairly confident about this
        (75% confidence). The Bayesian Network had the highest influence on this prediction
        (50% weight) because it is best at reasoning about weather and environmental
        factors. There is also a 30% chance of jam traffic conditions."
}
```

Running

```
# Install dependencies
pip install -r requirements.txt

# Optional: set TomTom key for live data
export TOMTOM_API_KEY=your_key_here

# Start all 5 services
bash run.sh
```

Services start in order: 4 model microservices (ports 8001–8004) → 5 s wait → ensemble orchestrator (port 8000).
Each service exposes `/health` and `/docs` (Swagger UI).
Stop all with `Ctrl-C`.

Dependencies

Package	Purpose
`fastapi` + `uvicorn`	HTTP microservice framework
`httpx`	Async HTTP client for inter-service calls + TomTom/Open-Meteo
`torch`	LSTM model
`scikit-learn`	Random Forest
`pgmpy`	Bayesian Network
`numpy`	Numerical ops shared across all models
`pydantic`	Request/response validation
`python-dotenv`	`.env` loading for `TOMTOM_API_KEY`